1. Why Incremental Loading?

Imagine an orders table already contains:

10 Million Records

Every day:

- New orders arrive
- Existing orders get updated
- Some statuses change

Example:

Yesterday:
Order 1003 → PENDING

Today:
Order 1003 → COMPLETED

Reloading all 10 million records every day just to process a few thousand changes may waste compute and processing time.

Instead, we can process only:

New Records
+
Changed Records

This is called **Incremental Loading**.

---

2. Full Load vs Incremental Load

Full Load

Processes the complete source dataset.

Example:

Source = 10 million records

Process all 10 million

Useful for:

- Initial ingestion
- Complete refresh
- Rebuilding a target

But it may become expensive as data grows.

Incremental Load

Processes only new or changed records.

Example:

100 New Orders
+
20 Updated Orders
=
120 Records Processed

Instead of reprocessing millions of unchanged records.

Remember

FULL LOAD
→ Process everything

INCREMENTAL LOAD
→ Process only new or changed data

Incremental loading also requires a reliable method to detect those changes.

---

3. What Is Delta MERGE?

`MERGE` compares incoming source records with records already present in a target Delta table.

Usually, we match using a **business key**.

For the orders dataset:

Business Key = order_id

Then:

Source matches Target
↓
UPDATE

Source doesn't match Target
↓
INSERT

This combination of:

UPDATE + INSERT

is called an:

UPSERT

---

4. Basic MERGE Structure

```
MERGE INTO target_table AS target
USING source_table AS source

ON target.order_id = source.order_id

WHEN MATCHED THEN
   UPDATE SET ...

WHEN NOT MATCHED THEN
   INSERT (...);
```

Important Parts

`ON`

Determines whether the source record already exists.

`WHEN MATCHED`

Handles existing records.

```
WHEN NOT MATCHED
```

Handles new records.

---

5. Target Table

For practice, the existing orders data is copied into:

datax_db.orders_incremental_target

```
CREATE OR REPLACE TABLE datax_db.orders_incremental_target
USING DELTA
AS
SELECT *
FROM datax_db.orders;
```

This represents the records already present in our production target.

Initial count:

10 Orders

---

6. Incoming Incremental Batch

Suppose the next batch contains five records.

Existing Orders Updated:
1003
1007

New Orders:
1011
1012
1013

Expected result:

2 Updates
+
3 Inserts

This small dataset represents the incremental data arriving from the source.

In production, incremental records might arrive through:

- Files
- Databases
- CDC systems
- Streaming pipelines

---

7. First Delta MERGE

```
MERGE INTO datax_db.orders_incremental_target AS target
USING datax_db.orders_incremental_source AS source

ON target.order_id = source.order_id

WHEN MATCHED THEN
UPDATE SET
   target.customer_id = source.customer_id,
   target.product_id = source.product_id,
   target.order_date = source.order_date,
   target.quantity = source.quantity,
   target.unit_price = source.unit_price,
   target.order_status = source.order_status

WHEN NOT MATCHED THEN
INSERT (
   order_id,
   customer_id,
   product_id,
   order_date,
   quantity,
   unit_price,
   order_status
)
VALUES (
   source.order_id,
   source.customer_id,
   source.product_id,
   source.order_date,
   source.quantity,
   source.unit_price,
```

```
    source.order_status
);
```

---

8. What Happens During MERGE?

For our batch:

1003 → Exists → UPDATE
1007 → Exists → UPDATE

1011 → New → INSERT
1012 → New → INSERT
1013 → New → INSERT

Before:

10 Records

Three new records are inserted:

13 Records

The two existing records are updated but do not increase the total record count.

---

9. Why MERGE Is Useful

Without MERGE, we might need separate logic for:

```
Check Record
     ↓
Exists?
 ↙      ↘
Yes       No
 ↓        ↓
UPDATE    INSERT
```

MERGE allows both paths to be handled together.

This is one reason it is commonly used in incremental pipelines.

---

10. What Is Idempotency?

Now imagine the same incremental batch gets processed twice.

Should we get duplicate orders?

With this MERGE design:

No

because records are matched using:

order_id

Existing records get updated instead of inserted again.

If the target has 13 rows after the first MERGE:

```
SELECT COUNT(*)
FROM datax_db.orders_incremental_target;
```

running the same MERGE again should still give:

13

---

11. Idempotency Definition

An operation is **idempotent** when repeating it produces the same intended final data state.

Example:

Run Batch Once
→ 13 records

Run Same Batch Again
→ 13 records

Important

MERGE is not automatically idempotent in every scenario.

Idempotency depends on things such as:

- Stable matching keys
- Deterministic logic
- Proper source deduplication
- Avoiding uncontrolled counters/timestamps in updates

---

12. Problem: Duplicate Records in the Source

Suppose incoming data contains:

order_id = 1003
status = PENDING
updated_at = 10:00

order_id = 1003
status = COMPLETED
updated_at = 11:00

Now two source records are trying to match the same target order.

Which one should be used?

If multiple source rows attempt to update the same target row, MERGE can become ambiguous and may fail.

So we should first decide which source record is correct.

---

13. Deduplicate Before MERGE

If we have:

updated_at

we can keep the latest record.

Use:

```
ROW_NUMBER() OVER (
   PARTITION BY order_id
   ORDER BY updated_at DESC
)
```

Then keep:

rn = 1

Example:

```
CREATE OR REPLACE TEMP VIEW orders_latest_source AS

SELECT
   order_id,
   order_status,
   updated_at
FROM (
   SELECT
      order_id,
      order_status,
      updated_at,

      ROW_NUMBER() OVER (
         PARTITION BY order_id
         ORDER BY updated_at DESC
      ) AS rn

   FROM orders_duplicate_source
)

WHERE rn = 1;
```

Now only the latest row remains for each order.

---

14. Example Deduplication Result

Before:

```
1003 → PENDING   → 10:00
1003 → COMPLETED → 11:00
1014 → PENDING   → 09:00
```

After deduplication:

1003 → COMPLETED → 11:00
1014 → PENDING → 09:00

Important

A timestamp may not always be enough.

If two records have the same timestamp, production logic may require another:

- Sequence number
- Event ID
- Reliable tie-breaker

---

15. Typical Incremental Pipeline Pattern

A practical Data Engineering pattern becomes:

Receive Batch
↓
Validate Data
↓
Deduplicate Source
↓
Identify Latest Record
↓
MERGE
↓
Update Existing Records
+
Insert New Records

This is a very common incremental pipeline design.

---

16. MERGE Logic Must Match the Source

Suppose your source contains only:

order_id
order_status

but the target requires:

order_id
customer_id
product_id
order_date
quantity
unit_price
order_status

You cannot blindly insert a complete new target record if required source columns are missing.

Your MERGE logic must match:

- Available source columns
- Target schema
- Business requirements

---

17. How Do We Identify Incremental Records?

MERGE applies changes.

But MERGE itself does **not automatically identify which source records changed**.

Something else must determine what records reach MERGE.

The class covers three common methods.

---

18. Method 1 — Timestamp-Based Incremental Loading

Suppose source records have:

updated_at

Track the last successful processing timestamp.

Example:

Last Run:
2026-09-08 10:00

Next Run:
Read records changed after 10:00

Things to handle carefully:

- Timestamp precision
- Overlapping windows
- Late-arriving changes

---

19. Method 2 — Change Data Capture (CDC)

Some source systems provide events such as:

INSERT
UPDATE
DELETE

Instead of scanning the complete source, the pipeline processes those change events.

CDC can provide more precise change tracking when configured correctly.

---

20. Method 3 — Incremental File Processing

Suppose the source delivers files:

orders_01.csv

orders_02.csv
orders_03.csv

Instead of reading every file again, process only files not previously ingested.

Databricks provides features such as:

Auto Loader

for this pattern.

---

21. What Is a Watermark?

In a batch incremental pipeline, a watermark can represent the last successfully processed point.

Examples:

Last Timestamp
Last Sequence Number

Conceptually:

Previous Watermark
↓
Read New Changes
↓
Process Successfully
↓
Store New Watermark

This batch concept is different from event-time watermarks used in Structured Streaming.

---

22. MERGE vs Change Detection

This distinction is important.

Change Detection
↓

Which records should we process?

MERGE
↓
How should those records be applied to the target?

So:

> MERGE applies changes.
> The ingestion strategy identifies those changes.

---

23. MERGE Is a Delta Transaction

MERGE is also recorded in Delta history.

Inspect:

DESCRIBE HISTORY datax_db.orders_incremental_target;

You may find:

MERGE

operations.

Depending on the runtime and operation, metrics may include counts such as:

- Rows inserted
- Rows updated
- Rows deleted

This can help validate production behavior.

---

24. Why MERGE Metrics Matter

Suppose your normal daily batch updates:

100 Records

but suddenly one run updates:

100,000 Records

That may indicate something unexpected.

MERGE history and metrics can help identify such changes.

---

25. Full Load vs Incremental Load — Quick Revision

Full Load	Incremental Load
Processes complete dataset	Processes new/changed records
Useful for initial load	Useful for recurring pipeline runs
Can require more processing	Avoids unnecessary reprocessing
Simpler change logic	Needs change-detection strategy

---

26. MERGE — Quick Revision

Source Record
↓
Match Target on Business Key
↓

Match Found?
↙ ↘
Yes No
↓ ↓
UPDATE INSERT

For this class:

Business Key
=

order_id

---

27. Common Mistakes

Mistake 1: Using an Unstable Matching Key

MERGE should use a reliable business key.

For example:

order_id

Mistake 2: Assuming MERGE Detects Changes

It does not.

MERGE only processes the source data given to it.

Mistake 3: Ignoring Source Duplicates

Multiple source rows for the same key may cause ambiguous updates.

Deduplicate first.

Mistake 4: Assuming MERGE Is Always Idempotent

It depends on deterministic logic and a stable matching strategy.

Mistake 5: Using Latest Timestamp Without Considering Ties

Two source rows may have the same timestamp.

A reliable tie-breaker may be required.

Mistake 6: Using `INSERT *` When Source Doesn't Contain Required Columns

MERGE logic must match the source and target schemas.

---

28. Student Practice Assignment

Use:

datax_db.orders_incremental_target

Task 1 — Create an Incremental Batch

Include:

Order 1005
→ Change status to CANCELLED

Order 1008
→ Change quantity from 4 to 5

Order 1015
→ New order

Order 1016
→ New order

Task 2 — Perform MERGE

Match using:

order_id

Then:

- Update existing records
- Insert new records
- Verify results

Task 3 — Test Idempotency

Run the same MERGE again.

Check:

```
SELECT COUNT(*)
FROM datax_db.orders_incremental_target;
```

Observe whether the count changes.

Explain why.

Task 4 — Source Deduplication

Create two records for:

order_id = 1015

with different:

updated_at

Use:

ROW_NUMBER()

to retain only the latest record.

Task 5 — Check Delta History

```
DESCRIBE HISTORY datax_db.orders_incremental_target;
```

Identify:

- MERGE operations

- Available operation metrics

---

29. Interview-Relevant Questions

1. What is incremental loading?

Incremental loading processes only new or changed data instead of repeatedly processing the complete dataset.

---

2. What is Delta MERGE?

MERGE applies conditional changes from a source dataset to a target Delta table.

It can update existing records and insert new records in one transaction.

---

3. What is an upsert?

UPSERT
=
UPDATE + INSERT

Existing records are updated while new records are inserted.

---

4. Why is source deduplication important before MERGE?

Because multiple source records matching the same target row can lead to ambiguous updates.

Deduplication helps select one correct source record per business key.

---

5. What is idempotency?

An operation is idempotent when repeating it produces the same intended final data state.

---

6. Is every MERGE automatically idempotent?

No.

It depends on:

- Stable keys
- Deterministic logic
- Proper source deduplication
- Update behavior

---

7. Does MERGE automatically detect which source records changed?

No.

MERGE applies the records supplied to it.

Change detection must happen through:

- Timestamps
- CDC
- Incremental files
- Another reliable ingestion mechanism

---

8. What is the matching condition in MERGE?

It determines whether the incoming source record already exists in the target.

Example:

ON target.order_id = source.order_id

---

9. What happens with `WHEN MATCHED`?

The source row already exists in the target, so we can update it.

---

10. What happens with `WHEN NOT MATCHED`?

The source row does not exist in the target, so we can insert it.

---

11. What is a watermark in a batch incremental pipeline?

It represents the last successfully processed point, such as:

- Timestamp
- Sequence number

and helps determine what data should be processed next.

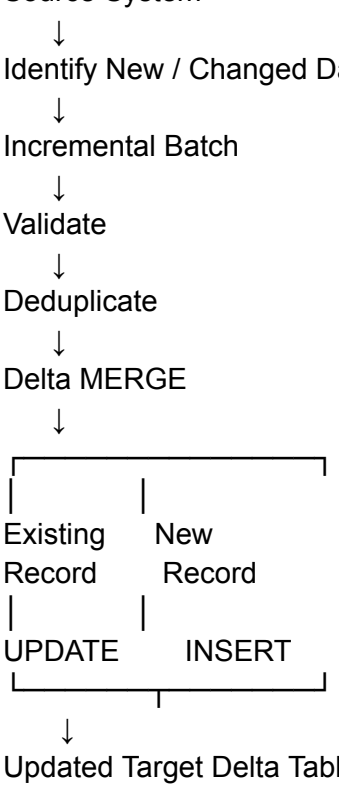
---

12. What is the difference between change detection and MERGE?

Change detection identifies **which records changed**.

MERGE determines **how those records are applied to the target table**.

Key Takeaway



The most important takeaway from this class is:

A reliable incremental pipeline is not just MERGE. It combines good change detection, stable business keys, source deduplication, idempotent processing, and Delta MERGE to safely apply changes to the target.